

clautolisp - Common Lisp AutoLISP Runtime & IDE - Detailed Architecture and Specification Draft

Converted from PDF

June 24, 2026

Contents

1	Executive Summary	1
2	Strategic Objectives	1
3	System Architecture Overview	2
4	alisp-core Runtime	2
4.1	Core Components	2
5	Visual LISP Compatibility Layer	2
6	Dialog Control Language Subsystem	3
7	Out-of-Process UI Architecture	3
8	CAD Host Integration	3
9	Snapshot-Based Regression Testing	3
10	IDE Capabilities	3
11	Migration Toward Common Lisp	4
12	Roadmap	4
13	Appendix A - Example Workflow	4
14	Appendix B - Future Enhancements	4

1 Executive Summary

This document describes the design of a portable AutoLISP runtime implemented in Common Lisp.

The goal is to allow legacy AutoLISP libraries to run offline outside CAD environments, while providing a modern development environment including debugging, testing, and regression validation.

The system is designed to run headless for automated testing and continuous integration pipelines. It also includes an optional UI layer implemented out of process using Qt6, enabling testable dialog logic without requiring a graphical environment.

2 Strategic Objectives

- Provide a standalone interpreter capable of executing existing AutoLISP libraries.
- Enable CAD regression testing through deterministic drawing snapshots.
- Provide a modern AutoLISP development environment with debugging, inspection, and testing tools.
- Allow progressive migration from AutoLISP toward Common Lisp.
- Ensure portability across macOS, Linux, and Windows.

3 System Architecture Overview

The system is composed of multiple subsystems organized around a layered architecture.

The interpreter core provides language semantics and runtime evaluation. Higher layers implement Visual LISP compatibility, UI abstraction, and host integration.

The modular architecture allows subsystems to evolve independently while maintaining stable interfaces.

Subsystem	Purpose	Estimated Public Functions
<code>alisp-core</code>	Language runtime and evaluator	120-200
<code>alisp-vl</code>	Visual LISP compatibility layer	180-350
<code>alisp-dcl</code>	Dialog control language subsystem	60-120
<code>alisp-host-*</code>	CAD integration and testing backend	80-180

4 `alisp-core` Runtime

The `alisp-core` subsystem implements the AutoLISP language semantics.

It includes the reader, evaluator, symbol system, environment model, and primitive functions.

The runtime preserves AutoLISP behavior including dynamic scoping and symbol resolution rules.

4.1 Core Components

- Reader: parses AutoLISP source expressions.
- Evaluator: executes forms using an explicit environment model.
- Environment model: stack of frames representing dynamic variable scope.
- Error system: mapped to Common Lisp conditions for debugging support.

5 Visual LISP Compatibility Layer

The Visual LISP compatibility layer provides helper utilities typically available in modern AutoLISP implementations.

This includes string processing functions, file utilities, list manipulation helpers, and conversion utilities.

The functions are grouped into categories such as `vl-string-*`, `vl-file-*`, and `vl-filename-*`.

6 Dialog Control Language Subsystem

The DCL subsystem provides support for dialog-based user interfaces.

It implements parsing of DCL files and runtime creation of dialogs.

Tiles map to platform widgets through the UI abstraction layer.

7 Out-of-Process UI Architecture

The UI layer is implemented as a separate process using Qt6.

This design enables the runtime to execute in headless environments while maintaining UI capabilities.

The runtime communicates with the UI process using a lightweight message protocol.

8 CAD Host Integration

The host subsystem provides bridges to external CAD environments.

Host backends translate AutoLISP entity operations into calls to the target CAD API.

A mock backend allows regression tests without requiring a CAD installation.

9 Snapshot-Based Regression Testing

Regression tests capture drawing state before and after AutoLISP execution.

Snapshots may be compared using multiple strategies including deep object comparison or entity counting.

This mechanism allows deterministic validation of AutoLISP libraries.

10 IDE Capabilities

The development environment integrates with SLIME for interactive debugging.

Developers can inspect runtime state, step through code, and analyze stack frames.

The IDE environment also provides regression testing tools and snapshot diff utilities.

11 Migration Toward Common Lisp

The interpreter allows gradual adoption of Common Lisp constructs.

Existing AutoLISP code can run unchanged while developers progressively adopt Common Lisp features.

This approach enables incremental modernization of legacy codebases.

12 Roadmap

- Phase 1: Core interpreter and REPL.
- Phase 2: Visual LISP compatibility layer.
- Phase 3: Testing harness and snapshot system.
- Phase 4: CAD host integrations.
- Phase 5: UI subsystem and IDE tools.

13 Appendix A - Example Workflow

1. Load an AutoLISP library into the interpreter.
2. Execute commands against a mock host backend.
3. Capture snapshot results.
4. Compare snapshots for regression validation.

14 Appendix B - Future Enhancements

- Graphical diff visualization for drawing changes.
- Integration with continuous integration pipelines.
- Advanced debugging tools including time-travel debugging.